

Exame Modelo 3 — Paradigmas de Programação — Época de Recurso 2025/2026

Instituição	P.PORTO — Escola Superior de Tecnologia e Gestão
Tipo de Prova	Exame Escrito — Época de Recurso
Curso	Licenciatura em Engenharia Informática / Licenciatura em Segurança Informática em Redes de Computadores
Unidade Curricular	Paradigmas de Programação
Ano Letivo	2025/2026
Duração	2 horas

Observações

- Não é permitida a consulta.
- Não são permitidas questões relativas à Parte 2. Sempre que considerarem necessário, os alunos devem assumir os pressupostos que entenderem adequados, indicando-os explicitamente na resolução.

Parte 1

Pergunta 1 (1,5 valores)

Explique o conceito de arrays de objetos em Java, distinguindo entre a criação do array e a criação dos objetos que o compõem. Descreva os cuidados necessários ao manipular arrays de objetos, nomeadamente a verificação de posições nulas. Explique também como se pode redimensionar um array e por que razão os arrays em Java têm tamanho fixo. Ilustre com um exemplo prático.

Pergunta 2 (1,5 valores)

Compare detalhadamente a utilização de `instanceof` com a utilização de `getClass()` para verificação de tipos em Java. Discuta as diferenças de comportamento na presença de herança e interfaces, explique em que contextos cada abordagem é mais adequada e fundamente com exemplos concretos que evidenciem as diferenças.

Pergunta 3 (1,5 valores)

Explique o conceito de membros estáticos (`static`) em Java, distinguindo entre atributos estáticos e métodos estáticos. Descreva como os membros estáticos se diferenciam dos membros de instância no que diz respeito ao ciclo de vida, ao acesso e à partilha de informação entre objetos. Ilustre com um exemplo prático que demonstre um caso de uso adequado para um atributo estático.

Pergunta 4 (1,5 valores)

Explique detalhadamente as diferenças entre classes abstratas e interfaces em Java. Em que situações é mais adequado optar por uma classe abstrata e em que situações é preferível uma interface? Justifique a sua resposta e ilustre cada caso com um exemplo prático que evidencie as características distintivas de cada mecanismo.

Parte 2

1. Considere as seguintes interfaces `Container` e `Measurement`. A interface `Container` descreve as operações de um contentor de bens utilizado numa instituição de ajuda humanitária. A interface `Measurement` descreve uma leitura de sensor.

```
public interface Container {  
    String getCode();  
    ItemType getType();  
    double getCapacity();  
    Measurement[] getMeasurements();  
    Measurement getLastMeasurement();  
    void addMeasurement(Measurement measurement) throws ContainerException;  
    boolean equals(Object obj);  
}
```

```
public interface Measurement {  
    double getValue();  
    String getDate();  
}
```

```
public enum ItemType {  
    PERISHABLE_FOOD,  
    NON_PERISHABLE_FOOD,  
    CLOTHING,  
    MEDICINE  
}
```

Pergunta 1a (3 valores)

Considere a interface `Container` que representa um contentor de bens. Implemente a interface numa classe denominada `ContainerImpl`. O contentor deve armazenar um array de medições (`Measurement`) com capacidade máxima de 50 medições. O método `addMeasurement` deve lançar `ContainerException` se a medição for nula ou se o valor da medição exceder a capacidade do contentor. O método `getLastMeasurement` deve devolver a última medição adicionada ou `null` se não existirem medições. Para o método `equals`, dois contentores são iguais se possuírem o mesmo código.

Pergunta 1b (2 valores)

Desenvolva o código necessário para testar a classe implementada (por exemplo, no contexto de um método `main`). Apresente um exemplo de teste para cada método implementado.

1. Considere a existência de uma classe `PickingMapImpl` que implemente a interface `PickingMap`.

```
public interface PickingMap {  
    AidBox[] getPickingMap(IIInstitution inst, ItemType type);  
}
```

```
public interface AidBox {  
    String getCode();  
    String getZone();  
    Container[] getContainers();  
}
```

Pergunta 2a (4 valores)

Implemente os seguintes métodos na classe `PickingMapImpl`:

```
boolean hasContainerOfType(AidBox aidbox, ItemType type);
```

- Este método deve devolver `true` caso a `AidBox` possua pelo menos um contentor cujo tipo corresponda ao `ItemType` recebido como argumento.

```
boolean needsCollection(Container container);
```

- Este método deve devolver `true` caso a última medição do contentor possua um valor superior a 70% da capacidade do contentor. Se o contentor for do tipo `PERISHABLE_FOOD`, deve devolver `true` se existir qualquer medição (independentemente do valor). Se o contentor não possuir medições, deve devolver `false`.

Pergunta 2b (5 valores)

Na classe `PickingMapImpl`, implemente o método `getPickingMap`.

Regras a considerar:

- O método deve percorrer todas as `AidBoxes` devolvidas pelo método `getAidBoxes()` da interface `IInstitution`.
- Apenas devem ser incluídas `AidBoxes` que possuam pelo menos um contentor do tipo especificado (utilizando `hasContainerOfType`).
- Das `AidBoxes` filtradas, apenas devem ser incluídas aquelas que possuam pelo menos um contentor do tipo especificado que necessite de recolha (utilizando `needsCollection`).
- O array devolvido não deve conter posições nulas.

Excertos de Código Fornecidos

```
public class InstitutionImpl implements IInstitution {  
    (...)  
    private AidBox[] aidBoxes;  
    private int numberOfAidBoxes;  
    private Vehicle[] vehicles;  
    private int numberOfVehicles;  
    (...)  
  
    public AidBox[] getAidBoxes() {  
        (...)  
    }  
  
    public Vehicle[] getVehicles() {  
        (...)  
    }  
}
```